

Software Engineering 2

Die ADHS-freundliche Klausur-Vorbereitung

29. Mai 2026 • HS A • 60 Min • 60 Punkte • Multiple Choice

IN 30 SEKUNDEN

Du hast morgen Klausur. Diese Zusammenfassung ist dein Survival-Kit:

- **Blaue Boxen** (wie diese) = die Hauptaussage in 30 Sekunden
- **Orange Boxen** = Stories & Analogien (damit's hängenbleibt)
- **Grüne Boxen** = Muss-Wissen Fakten
- **Gelbe Boxen** = Eselsbrücken & Merkhilfen
- **Rote Boxen** = Klausur-Fallen, wo Punkte verloren gehen
- **Lila Boxen** = Quick-Checks, teste dich

Notenschlüssel: $5 < 36 \leq 4 < 42 \leq 3 < 48 \leq 2 < 54 \leq 1$

Faustregel: Du brauchst $\geq 60\%$ richtig (36 Punkte) für ein **4er**.

Inhaltsverzeichnis

1 Was ist Software Engineering überhaupt?	5
1.1 Hacker vs. Software Engineering – die Tabelle die in der Klausur vorkommen KÖNNTE	5
1.2 Die 4 Merkmale guter Software	5
2 Softwareentwicklungs-Prozesse	7
2.1 Wasserfall (Royce 1970)	7
2.2 V-Modell	7
2.3 Spiral-Modell	8
2.4 Agile Modelle	8
2.5 Agile vs. Klassische Modelle – die Tabelle die DEFINITIV vorkommt	9
3 SCRUM – der Klausur-Liebling	10
3.1 Die 3 Rollen	10
3.2 Das Product Backlog	11
3.3 Kano-Modell (Kundenzufriedenheit)	11
3.4 User Stories & INVEST	11
3.5 Sprint – der Herzschlag	12
3.6 Die 4 Meetings	12
3.7 Schätzen – Story Points & Planning Poker	13
3.8 Velocity	13
3.9 Burndown Chart	13
3.10 Definition of Done (DoD)	13
4 Software Design	15
4.1 Schichtenarchitektur	15
4.2 MVC vs. MVP – der Klassiker	15
4.3 Komponenten & Services	16
4.4 Dependency Injection (DI)	16
4.5 AOP – Aspektorientierte Programmierung	17
4.6 Web Services: REST vs. SOAP	17
4.7 Microservices	17
5 Anforderungen (Requirements Engineering)	19
5.1 Die 5 Aufgaben des RE – MERKEN!	19
5.2 Stakeholder	19

5.3	FA vs. NFA – die wichtigste Unterscheidung	19
5.4	Die 9 Qualitätskriterien für Anforderungen	20
5.5	Fehler in Anforderungen – die Beispiele aus der Vorlesung	20
5.6	Erhebung – die richtige Technik wählen	20
5.7	Doku: Lastenheft vs. Pflichtenheft	21
5.8	Dokumentations-Methoden	21
5.9	Validierung – 3 Review-Typen	22
6	Qualitätssicherung (QS)	23
6.1	Die 8 Qualitätsfaktoren (ISO 25010)	23
6.2	Analytische QS	23
6.3	Software-Fehler-Begriffe – WICHTIG für MC!	24
6.4	Software-Testen	24
6.5	GQM (Goal-Question-Metric)	24
6.6	CMMI – die 5 Reifegrade	25
6.7	Standards	25
7	Projektmanagement	26
7.1	Das magische Dreieck (eigentlich 4eck!)	26
7.2	Projektphasen	26
7.3	Organisationsformen	26
7.4	Schätzverfahren	26
7.5	Delphi-Methode	27
7.6	Function Points (Albrecht 1979, IBM)	27
7.7	COCOMO 81 (Boehm 1981)	27
7.8	Planung: WBS, PERT, GANTT	28
7.9	Risikomanagement	28
7.10	Personal-Mythos – Brooks' Law	29
8	Konfigurationsmanagement (SCM)	30
8.1	Änderungsmanagement	30
8.2	Versionsmanagement – Git	30
8.3	Gute Commit Messages	31
8.4	Tangled Commits vermeiden	31
8.5	Merging-Strategien	31
8.6	Semantic Versioning	31
8.7	Software Automation Pipeline	32
8.8	DevOps – Wall of Confusion	32

9	Notfall-Cheatsheet – die Top-30 Fakten	34
9.1	Zitate & Personen	34
9.2	Schlüssel-Akronyme	34
9.3	Zahlen die du wissen MUSST	35
10	Letzte Klausur-Tipps	36

1. Was ist Software Engineering überhaupt?

IN 30 SEKUNDEN

SE = die **Kunst**, große Software-Systeme **wirtschaftlich, schnell und zuverlässig** zu bauen und am Leben zu halten.

DIE STORY

Stell dir vor, du musst einen **Wolkenkratzer** bauen. Du kannst nicht einfach drauflos legen. Du brauchst Pläne, Statiker, Bauleiter, Qualitätskontrolle, Wartung.

Software ist genauso: **Hacker-Style** (zu Hause an einem Skript basteln) $\neq$ **Industrie-Software** (Bank-System, Krankenhaus, Auto).

1.1 Hacker vs. Software Engineering – die Tabelle die in der Klausur vorkommen KÖNNTE

Personal Software (Hacken)	vs. Industrial Software (SE)
Entwickler = Nutzer	Kunde = Nutzer
Bugs sind tolerabel	Bugs werden nicht toleriert
UI egal	UI wichtig
Wenig/keine Doku	Viel Doku
Nicht kritisch	Unterstützt Geschäftsprozesse
Robustheit egal	Robustheit kritisch
Keine Investitionen	5–25 \$ pro LOC !
Portabilität egal	Portabilität = Geld

1.2 Die 4 Merkmale guter Software

MUSS-WISSEN

1. **Wartbarkeit** – Kann ich sie ändern, ohne dass alles bricht?
2. **Sicherheit & Zuverlässigkeit** – Crasht sie nicht? Hackt sie keiner?
3. **Effizienz** – Frisst sie nicht den ganzen Speicher?
4. **Benutzerfreundlichkeit** – Versteht sie ein Mensch?

MERKHILFE

„Warum Sicher Effizient Benutzbar?“ = **W-S-E-B**

KLAUSUR-FALLE

SE ist NICHT dasselbe wie Informatik!

- Informatik = Theorien und Methoden *der Software-Systeme*
- SE = Probleme der *Herstellung* von Software

Wenn du das im MC siehst: **Unterschied!** Häufige Falle.

2. Softwareentwicklungs-Prozesse

IN 30 SEKUNDEN

Verschiedene **Vorgehensweisen**, um Software zu bauen.

Sequenziell (Wasserfall, V): geplant, starr

Iterativ (Spiral): runden-basiert, mit Risikoanalyse

Agil (Scrum, XP, Kanban): kurze Zyklen, flexibel

2.1 Wasserfall (Royce 1970)

DIE STORY

Wasser fließt nur nach unten, nie zurück. Genauso die Phasen: Anforderungen → Design → Code → Test → Wartung. *Reihenfolge fix*.

Problem: Wenn der Kunde am Ende sagt „*Ich wollte eigentlich was anderes*“ – pech gehabt.

MUSS-WISSEN

- **Top-Down**, festgelegte Reihenfolge
- **Eingeschränkte Rückkopplung**
- Benutzerbeteiligung **nur zu Beginn**

(+) (theoretisch) gut planbar, geringer Mgmt-Aufwand

(-) Kurskorrekturen zu spät, starr, Doku wird wichtiger als das System

2.2 V-Modell

DIE STORY

Wasserfall trifft Test-Schwester. Für jede Bauphase **links** gibt's rechts eine **Test/Validation**-Phase. Bildet ein **V**.

MUSS-WISSEN

- **Qualitätssicherung integriert**
- Sehr umfangreich – für **große Projekte**
- **Verifikation vs. Validation:**
 - **Verifikation** = „Bauen wir das Produkt *richtig*?“ (gegen Spec)
 - **Validation** = „Bauen wir das *richtige* Produkt?“ (Kunde happy?)

(+) integriert detailliert, generisch, gut bei großen Projekten

(-) Bürokratie bei kleinen Projekten, „Big Bang“ am Schluss, starr

MERKHILFE

Verifikation = „Verify against Spec“ (Spezifikation)

Validation = „Valid for the Customer“ (Kunde happy?)

2.3 Spiral-Modell

DIE STORY

Stell dir eine **Spirale** vor, die nach außen wächst. Jede Runde:

1. Ziele/Einschränkungen festlegen
2. **Risiken analysieren** (das Herzstück!)
3. Entwickeln (Design, Code, Test)
4. Nächste Runde planen

MUSS-WISSEN

Rundenbasiert. Risiko zuerst checken, dann entwickeln. Gut für große Systeme mit mehreren Disziplinen.

(+) Risiken früh, anpassungsfähig, inkrementell

(-) Mehrarbeit, komplexes PM, theoretisch

2.4 Agile Modelle

DIE STORY

Gegenbewegung zu starren Prozessen. Idee: „Wir wissen nicht, was der Kunde am Ende wirklich will. Also bauen wir kurze Iterationen und passen ständig an.“

MUSS-WISSEN

Das Agile Manifest – 4 Werte:

1. **Individuen & Interaktionen** > Prozesse & Tools
2. **Funktionierende Software** > ausführliche Doku
3. **Kundenkooperation** > Vertragsverhandlung
4. **Reagieren auf Veränderung** > Plan befolgen

Charakteristika:

- Zyklen: 2 Wochen bis einige Monate
- Kleine Teams: **6–8 Personen**
- Wenig Doku
- Kunde-Präsenz wichtig
- Keine Dogmen

2.5 Agile vs. Klassische Modelle – die Tabelle die DEFINITIV vorkommt

Aspekt	Klassisch	Agil
Kundenmitwirkung	Unwahrscheinlich	Kritischer Erfolgsfaktor
Lieferung	Erst nach langer Zeit	Alle 6 Wochen
Richtiges Produkt durch	Langes Spezifizieren	Kern, zeigen, verbessern
Disziplin	Formal, wenig	Informell, viel
Änderungen	Widerstand	Werden erwartet
Kommunikation	Über Dokumente	Zwischen Menschen
Änderungen vorsorgen durch	Vorausplanung	Flexibel bleiben

KLAUSUR-FALLE

Es gibt KEIN bestes Modell! Beliebte Frage in der Klausur.

Antwort: „Das beste Modell hängt von Organisation, Produkt und Entwickler-Fähigkeiten ab.“

CHECK DICH SELBST

Was ist der Hauptunterschied zwischen Wasserfall und Spiralmodell?

Antwort: Wasserfall = sequenziell, einmal durch. Spiral = iterativ, mit **Risikoanalyse pro Runde**.

3. SCRUM – der Klausur-Liebling

IN 30 SEKUNDEN

SCRUM ist NICHT mehr und NICHT weniger als ein Rahmen, in dem ein selbstorganisierendes Team in kurzen Sprints (1 Monat) Software liefert.

Begriff aus dem Rugby: Team-Formation, klares Ziel, freie Taktik.

DIE STORY

Stell dir ein Rugby-Team vor. Der Coach sagt: „*Eure Aufgabe ist: Touchdown.*“ Wie? Das entscheidet das Team selbst auf dem Feld. Genauso bei SCRUM: *Product Owner* sagt WAS, *Team* entscheidet WIE.

3.1 Die 3 Rollen

MUSS-WISSEN

1. Team (5–10 Vollzeit!)

- Heterogen (Dev, QA, UI), aber alle machen alles
- Selbstorganisierend, keine Titel
- Verantwortlich für **Sprint-Erfolg (Commitment)**

2. Scrum Master

- **Schützt das Team** vor äußeren Störungen
- **Entfernt Hindernisse**
- Achtet auf Einhaltung der Scrum-Werte
- *Bei DoD-Verletzung:* ER ist zuständig!

3. Product Owner

- Repräsentiert den **Kunden**
- Verantwortlich für **ROI**
- **Priorisiert** das Product Backlog
- Akzeptiert/lehnt Arbeitsergebnisse ab

MERKHILFE

Team – macht

Scrum Master – räumt

Product Owner – sagt

3.2 Das Product Backlog

MUSS-WISSEN

- Liste **aller Projektarbeiten**, vom Product Owner priorisiert
- Je weiter oben, desto **genauer** spezifiziert
- Wird **laufend** ergänzt und verfeinert
- Jedes Item = eine **User Story**

3.3 Kano-Modell (Kundenzufriedenheit)

DIE STORY

Du kaufst dir ein Auto. Welche Features machen dich glücklich?

- **Basis** = Sicherheit, Rostschutz („*müssen halt sein*“)
- **Begeisterung** = Sonderausstattung, Design („*wow!*“)
- **Leistung** = Beschleunigung, Verbrauch („*je mehr, desto besser*“)
- **Unerheblich** = Farbe? Automatik? („*whatever*“)
- **Rückweisung** = Dellen, keine Zulassung („*wenn das, dann kauf ich's NICHT*“)

MERKHILFE

Gewöhnungseffekt: Begeisterung wird mit der Zeit zu Basis.
Beispiel: SMS war einmal begeisternd, heute ein Muss.

3.4 User Stories & INVEST

MUSS-WISSEN

Format: „Als *<Rolle>* will ich *<tun>*, so dass ich *<Grund>*.“

INVEST:

- Independent – unabhängig
- Negotiable – verhandelbar (nicht im Sprint!)
- Valuable – Mehrwert
- Estimateable – schätzbar
- Small – konkret/klein
- Testable – testbar (Akzeptanzkriterien!)

MERKHILFE

INVEST = „*Investiere Zeit in eine gute Story*“.
Akronym von oben nach unten lesen.

3.5 Sprint – der Herzschlag

MUSS-WISSEN

- Dauer: **ca. 1 Monat** ($\pm$ 1–2 Wochen), konstant
- Im Sprint wird **entworfen, kodiert UND getestet**
- **KEINE Änderungen** während Sprints – sonst Sprint-Ende
- **KEINE Einschränkungen** für das Team

3.6 Die 4 Meetings

IN 30 SEKUNDEN

Sprint Planning (Anfang) → **Daily Scrum** (täglich) → **Sprint Review** (Ende) → **Retrospektive** (Ende-Reflexion)

MUSS-WISSEN

Sprint Planning: Was kommt in den nächsten Sprint? Output: **Sprint Goal + Sprint Backlog**

Daily Scrum:

- **15 Minuten**, täglich, Stand-up
- **Nicht zur Problemlösung** (das danach!)
- 3 Fragen: Was getan? Was tun? Was hindert?
- **Hühner und Schweine nehmen teil – nur Schweine reden**

Sprint Review: Team zeigt Demo. **Informell** („2h-Vorbereitung“-Regel). Teilnehmer: Kunde, Mgmt, PO, Team.

Retrospektive: Nur Team! Start/Continue/Stop. **Keinesfalls Performance-Review!**

DIE STORY

Hühner und Schweine? Witz: Ein Huhn und ein Schwein wollen ein Restaurant aufmachen: „Ham and Eggs“. Schwein: „*Moment – du bist nur **beteiligt**, ich bin **committed**!*“

→ Im Daily reden nur die, die committed sind (das Team).

KLAUSUR-FALLE

Häufige Fallen im MC:

- Daily Scrum „*kann durch E-Mail ersetzt werden*“ → **FALSCH!**
- Retrospektive „*mit Management*“ → **FALSCH!** (nur Team)
- Review „*sehr formell, lange Vorbereitung*“ → **FALSCH!** (informell, 2h)

3.7 Schätzen – Story Points & Planning Poker

MUSS-WISSEN

- **Was wird geschätzt?** Aufwand pro Story *im Vergleich*
- **Wer schätzt?** Das Team!
- **Wie?** Story Points + Planning Poker
- Größenordnungen: **1, 2, 3, 5, 8, 13, 20, 40, 100** (Fibonacci-artig)
- **> 13 Story Points = EPIC** → muss vor Sprint geteilt werden!

MERKHILFE

Story Points $\neq$ **Personentage!** Es sind *relative* Größen, keine Stunden.

3.8 Velocity

MUSS-WISSEN

Velocity = wie viele Story Points schafft das Team pro Sprint?

- Bei Urlaub/Krankheit nicht voll ausschöpfbar
- **Nur 100% fertige Stories zählen** – 80% = 0 Wert für Kunden
- Berechnung: Bauchgefühl (1. Sprint), dann „Übernommen“ oder **Median**

3.9 Burndown Chart

MUSS-WISSEN

- Zeigt **verbleibenden Aufwand pro Tag**
- Täglich aktualisiert
- **Bei Missbrauch** (z.B. Mgmt-Review) → **ABSCHAFFEN!**

3.10 Definition of Done (DoD)

MUSS-WISSEN

- **Wann ist etwas „fertig“?**
- Sammlung an Bedingungen, **messbar**
- **Team-/projektspezifisch**
- **Verletzung** der DoD → **Scrum Master** ist zuständig

CHECK DICH SELBST

Q: Was passiert, wenn während eines Sprints ein Produktions-Notfall auftritt?

A: Bei **sofortiger Notwendigkeit** → **Sprint-Ende** und Reparatur. Sonst geplant einreihen.

Q: Wie viele Personen hat ein Scrum-Team?

A: **5–10** (in der Vorlesung).

4. Software Design

IN 30 SEKUNDEN

Wie strukturierst du Code so, dass er nicht zum unwartbaren Spaghetti-Monster wird?

Antwort: **Schichten** (Layers), **Patterns** (MVC/MVP) und **Reuse** (Komponenten/Services).

4.1 Schichtenarchitektur

DIE STORY

Wie eine Lasagne. Oben die hübsche Käse-Schicht (UI), darunter die Logik, ganz unten die Daten. Jede Schicht spricht nur mit der direkt darunter.

MUSS-WISSEN

Typische Schichten:

1. **Presentation** – UI
2. **Application** (optional) – Koordination
3. **Business** – Logik, Validierung
4. **Technical** – DB, Hardware

(+) unabhängig, austauschbar, leicht verständlich

(–) Performance (durch alle Schichten – Lösung: **Layer Bridge**), Änderungen in mehreren Schichten, Skalierbarkeit

4.2 MVC vs. MVP – der Klassiker

DIE STORY

Beide: Trenne Daten / Anzeige / Logik.

Unterschied: In MVP ist die View ein „dummes“ Anzeige-Ding, der Presenter ist der Mittelsmann.

MUSS-WISSEN

MVC (Model-View-Controller):

- **Model** – Daten/Status, Anwendungslogik
- **View** – Anzeige
- **Controller** – interpretiert Input, triggert Model

MVP (Model-View-Presenter):

- **Model** – wie MVC
- **View (PASSIV)** – nur Anzeige
- **Presenter** – **Mittelsmann** zwischen View und Model

MERKHILFE

MVP = bessere Testbarkeit! Weil die View komplett dumm ist, kann man den Presenter ohne UI testen.

4.3 Komponenten & Services

DIE STORY**Reuse-Pyramide:**

- **Direkter Aufruf** = sofort, aber starr (`new WordImport()`)
- **Interface** = flexibler, aber Code-Recompile
- **Komponente** = klare Schnittstelle, locker verbunden
- **Service** = übers Netzwerk, plattformunabhängig

MUSS-WISSEN**Komponenten-Eigenschaften (Szyperski):**

1. Standardisiert/dokumentiert
2. Unabhängig (deployable)
3. Zusammensetzbar
4. Verteilbar

Service-Anforderungen:

- **Zustandslos**, lose Kopplung
- Service Registry (Verzeichnis)
- Ortstransparenz, plattformunabhängig

4.4 Dependency Injection (DI)

DIE STORY

Statt: „*Ich hole mir mein Auto selber aus der Garage.*“

DI: „*Das Framework gibt mir das Auto, wenn ich's brauche.*“

→ Du kennst nur das **Interface** („Auto“), nicht das konkrete Modell.

MUSS-WISSEN

- **Pattern für Inversion of Control**
 - Client kennt nur die Schnittstelle
 - Konfiguration: XML **oder** Annotations (`@Autowired`)
 - Beans als **Singleton**
 - Varianten: **Setter Injection, Constructor Injection**
- (+) bessere Strukturierung, Testbarkeit (Mocking!), Wiederverwendbarkeit
(-) Komplexität, Einarbeitung

4.5 AOP – Aspektorientierte Programmierung

DIE STORY

„**Cross-Cutting Concerns**“ sind Dinge, die du *überall* brauchst: **Logging, Security, Transaktionen**. Statt sie in jede Klasse zu schreiben, definierst du **Aspekte**, die das Framework automatisch einbindet.

MUSS-WISSEN

Begriffe:

- **Aspekt** – Klasse mit Querschnittsfunktion
- **Advice** – Implementierung (Before/After/Around)
- **Join Point** – Punkt, an dem Aspekte greifen können
- **Pointcut** – Gruppe von Join Points

4.6 Web Services: REST vs. SOAP

REST (modern, einfach)	XML/SOAP (alt, formal)
HTTP-basiert, URIs	RPC-artig, XML-basiert
JSON, CSV, XML	nur XML
GET, POST, PUT, DELETE	SOAP-Envelope
Keine Standard-Beschreibung	WSDL beschreibt
Kein Verzeichnis	UDDI als Registry

MERKHILFE

REST-HTTP-Methoden = CRUD:

Create = POST, **Read** = GET, **Update** = PUT, **Delete** = DELETE

SOAP-Aufbau = „Envelope, Header, Body, Fault“

→ Stell dir einen Brief vor: Umschlag, Absender, Inhalt, Fehlermeldung

4.7 Microservices

DIE STORY

Statt ein riesiger Monolith, ist die App eine **Sammlung kleiner Services**, die jeweils eine Sache gut können. Beispiel: Amazon.

MUSS-WISSEN

Eigenschaften: lose gekoppelt, leicht wartbar, einzeln testbar, unabhängig zusammensetzbar, austauschbar.

CHECK DICH SELBST

Q: Was ist der Hauptvorteil von MVP gegenüber MVC?

A: **Bessere Testbarkeit** (View ist passiv).

Q: Wofür steht REST?

A: **Representational State Transfer**.

5. Anforderungen (Requirements Engineering)

IN 30 SEKUNDEN

Bevor du baust, musst du wissen WAS gebaut werden soll.

RE = systematisch herausfinden, was die Stakeholder wirklich brauchen.

DIE STORY

Chaos Report: Top 3 Gründe warum Projekte scheitern:

1. Unvollständige Anforderungen (13%)
2. Kunden nicht einbezogen (12%)
3. Mittel nicht ausreichend (10%)

→ **Anforderungen sind der wichtigste Schritt!**

5.1 Die 5 Aufgaben des RE – MERKEN!

MUSS-WISSEN

Ermitteln – Dokumentieren – Prüfen – Abstimmen – Verwalten

MERKHILFE

„**E-D-P-A-V**“ – *Endlich Doku, Prüfen, Abstimmen, Verwalten*

5.2 Stakeholder

MUSS-WISSEN

Personen oder Organisationen mit **direktem oder indirektem Einfluss** auf das System: Anwender, Kunde, Management, Servicepersonal, Product Owner, Marketing, Schulungspersonal...

5.3 FA vs. NFA – die wichtigste Unterscheidung

IN 30 SEKUNDEN

FA = WAS (Features)

NFA = WIE GUT (Qualität)

MUSS-WISSEN

Funktionale Anforderung (FA):

- Beispiel: „Die Suche soll alle Bücher zu einem Thema liefern.“

Nicht-Funktionale Anforderung (NFA):

- Performance, Security, Usability, Reliability...
- Beispiel: „Das Suchergebnis nach max. 5 Sekunden.“
- **Oft kritischer als FA**, betrifft **gesamte Architektur**

5.4 Die 9 Qualitätskriterien für Anforderungen

MUSS-WISSEN

Eine gute Anforderung ist:

1. **vollständig**
2. **atomar** – genau eine Anforderung pro Satz
3. **technisch lösungsneutral** – WAS, nicht WIE
4. **notwendig**
5. **verfolgbar**
6. **realisierbar**
7. **konsistent**
8. **eindeutig**
9. **prüfbar** (testbar)

5.5 Fehler in Anforderungen – die Beispiele aus der Vorlesung

MUSS-WISSEN

- „Zugtüren immer geschlossen **ZWISCHEN** Stationen. Sie müssen offen sein **WENN** Nothalt.“ → **WIDERSPRUCH**
- „Türen öffnen, wenn Zug in Station.“ → **MEHRDEUTIG** (sofort? mit Knopf?)
- „Anzeige muss benutzerfreundlich sein.“ → **UNMESSBAR**
- „Jedes Abteil hat Info-Panel und Nichtraucher-Aufkleber.“ → **NOISE** (irrelevant)
- „Anzeige ≥ 7 km/h schneller als real.“ → **UNDURCHSICHTIG**

5.6 Erhebung – die richtige Technik wählen

DIE STORY

Drei Wissens Ebenen brauchen drei Strategien:

- **Bewusstes Wissen** (kann erzählt werden) → **Befragen**
- **Unbewusstes Wissen** (muss geweckt werden) → **Kreativität**

- **Unterbewusstes Wissen** (Stakeholder weiß es nicht) → **Beobachten**

MUSS-WISSEN

Kreativität (Begeisterungs): Brainstorming, 6-3-5, Wechsel der Perspektive (6 Hüte)

Beobachtung (Basis): Feldbeobachtung, Apprenticing

Befragung (Leistung): Fragebogen, **Interview** (wichtigste!)

Artefakt-basiert: Hintergrundstudie, Systemarchäologie, Reuse

Unterstützend: Workshop, Mind Mapping, Audio/Video, Szenarien

MERKHILFE

6-3-5 Methode: 6 Teilnehmer, je 3 Ideen, 5-mal weitergeben.

Apprenticing („in die Lehre gehen“): Du bist der **Lehrling**, der Stakeholder ist der **Meister**. NICHT für Flugsicherung etc.!

5.7 Doku: Lastenheft vs. Pflichtenheft

IN 30 SEKUNDEN

Lastenheft = vom Kunden (WAS will ich)

Pflichtenheft = vom Auftragnehmer (WIE setze ich's um)

MERKHILFE

Lastenheft = „Last“ = Auftraggeber lädt das Projekt ab.

Pflichtenheft = „Pflicht“ = Auftragnehmer verpflichtet sich zur Umsetzung.

5.8 Dokumentations-Methoden

MUSS-WISSEN

- **Prosa** – natürliche Sprache (häufigste; aber mehrdeutig)
- **User Stories** – agil
- **Use Case Diagramm** – Übersicht (keine NFAs!)
- **Use Case Beschreibung** – detailliert mit Template
- **Aktivitätsdiagramm** – Workflow mit Entscheidungen
- **Sequenzdiagramm** – zeitliche Abfolge

5.9 Validierung – 3 Review-Typen

MUSS-WISSEN

Aufwand wächst: Stellungnahme < Walkthrough < Inspektion

Stellungnahme: Kollege liest, markiert Auffälligkeiten

Walkthrough: Moderator + Autor präsentiert + Prüfer

Inspektion: Formal, **6 Phasen**, Checklisten, auch auf Code/Testfälle anwendbar

MERKHILFE

Inspektions-Phasen (6):

1. Planung
2. Vorbesprechung
3. Individuelle Vorbereitung
4. Reviewsitzung
5. Nachbereitung/Bewertung
6. Ende

KLAUSUR-FALLE

Häufige Klausur-Falle: „Inspektion ist nur für Anforderungen.“

FALSCH! Inspektion geht auch auf **Testfälle, Code** etc.

CHECK DICH SELBST

Q: Was sind die 5 Aufgaben des RE?

A: **Ermitteln, Dokumentieren, Prüfen, Abstimmen, Verwalten.**

Q: Was bedeutet **atomar**?

A: Genau **eine** Anforderung pro Satz. Keine Konjunktionen.

Q: Lastenheft vs. Pflichtenheft?

A: **Lasten** = Kunde, **WAS**. **Pflichten** = Auftragnehmer, **WIE**.

6. Qualitätssicherung (QS)

IN 30 SEKUNDEN

Wie stellen wir sicher, dass die Software nicht Schrott ist?

Antwort: 3 Methoden = **Analytisch** (prüfen), **Konstruktiv** (vermeiden), **Organisatorisch** (Standards)

6.1 Die 8 Qualitätsfaktoren (ISO 25010)

MUSS-WISSEN

- **Functionality** – macht's was es soll?
- **Reliability** – ist's zuverlässig?
- **Usability** – ist's benutzbar?
- **Efficiency** – ist's schnell?
- **Maintainability** – ist's wartbar?
- **Portability** – läuft's woanders?
- **Security** (NEU in 25010) – ist's sicher?
- **Compatibility** (NEU in 25010) – spielt's mit anderen?

MERKHILFE

„**F-R-U-E-M-P-S-C**“ oder einfach merken:

Funkt's Richtig, ist's nutzbar, schnell, wartbar, portabel, sicher, kompatibel?

Achtung: Security und Compatibility kamen 2011 dazu.

6.2 Analytische QS

MUSS-WISSEN

Statisch (ohne Code-Ausführung):

- **Metriken** (mit GQM-Ansatz)
- **Reviews** (Stellungnahme/Walkthrough/Inspektion)

Dynamisch (mit Ausführung):

- Software Testen
- **Verifikation vs. Validation** (wie V-Modell)

6.3 Software-Fehler-Begriffe – WICHTIG für MC!

MUSS-WISSEN

IEEE 610.12 Definitionen:

- **Software Error (Fehler)** = menschliche Aktion mit inkorrektem Resultat
- **Software Fault (Fehlerfall)** = inkorrektter Schritt im Programm/Daten
- **Software Failure (Fehlverhalten)** = Unfähigkeit des Systems

MERKHILFE

E-F-F: Error (Mensch) → Fault (Code) → Failure (System)

Geschichte: Der Mensch macht einen Fehler (Error), der einen Fault im Code hinterlässt, der zu einem Failure führt, wenn ausgeführt.

6.4 Software-Testen

DIE STORY

Dijkstra: „Testing can show the presence of bugs, but not their absence.“

→ Tests beweisen NICHT, dass Bugs fehlen. Nur, dass welche da sind.

MUSS-WISSEN

- **Black-Box** – ohne Code-Kennntnis
- **White-Box** – mit Code-Kennntnis
- **Äquivalenzklassen**
- **Grenzwertanalyse**
- **Mocking**
- **Test-Coverage:** Statement, Branch, einfache Bedingung, mehrfache Bedingung, Path

6.5 GQM (Goal-Question-Metric)

MUSS-WISSEN

Wie findet man die richtigen Metriken?

1. **Goals** – Ziele definieren
2. **Questions** – Fragen ableiten
3. **Metrics** – Metriken festlegen

6.6 CMMI – die 5 Reifegrade

DIE STORY

Vom Chaos zur Perfektion – in 5 Stufen.

MUSS-WISSEN

1. **Initial** – ad-hoc, „irgendwie“, Erfolg unerklärbar
2. **Repeatable** – Planung, aber Erfolg hängt von *Personen* ab
3. **Defined** – institutionalisierter Prozess, Qualität meist gut
4. **Managed** – durch **Messen** gesteuert, vorhersagbar hoch
5. **Optimizing** – kontinuierliches Feedback zur Verbesserung

MERKHILFE

„**I-R-D-M-O**“ = *Initial* → *Repeatable* → *Defined* → *Managed* → *Optimizing*
Eselsbrücke: „*I’m Really Determined to Manage Optimally*“

KLAUSUR-FALLE

Falle: Welche Stufe hängt von Personen ab? → **Stufe 2 (Repeatable)**
Auf Stufe 3 (Defined) ist der Prozess *institutionalisiert*.

6.7 Standards

MUSS-WISSEN

- **ISO 9000/9001** – Qualitätsmgmt
- **ISO/IEC 25010** – Qualitätsfaktoren
- **IEEE 828** – SCM
- **IEEE 1061** – Metriken
- **CMMI** – Reifegrad (SEI = Carnegie Mellon)

CHECK DICH SELBST

Q: Wie heißt die CMMI-Stufe 3?

A: **Defined**.

Q: „Bauen wir das Produkt richtig?“ ist welche Frage?

A: **Verifikation**.

Q: Was sagt Dijkstra über Testing?

A: Tests zeigen Bugs, beweisen aber NICHT deren Abwesenheit.

7. Projektmanagement

IN 30 SEKUNDEN

Wie steuere ich ein Projekt erfolgreich?

3 Hauptthemen: **Aufwand schätzen, planen+steuern, Personal+Risiken managen.**

7.1 Das magische Dreieck (eigentlich 4eck!)

MUSS-WISSEN

Zeit – Kosten – Qualität – Funktionalität

→ Du kannst nicht alles gleichzeitig maximieren.

7.2 Projektphasen

MUSS-WISSEN

Definition → Planung → Verfolgung → Abschluss

(Unabhängig von Organisationsform!)

7.3 Organisationsformen

MUSS-WISSEN

- **Stab-Projekt:** Abteilungen mit Spezialisierungen
- **Reine Projekt:** MA werden „abgezogen“
- **Matrix:** Balance der beiden (MA haben 2 Chefs)

7.4 Schätzverfahren

DIE STORY

Schätztrichter (Böhm 2000): Je näher dem Projektende, desto **genauer** die Schätzung. Anfangs riesiger Fehler, am Ende klein.

MUSS-WISSEN

3 Kategorien:

1. **Empirisch** (Erfahrung): Experten, Delphi, Analogie, Bottom-Up
2. **Algorithmisch** (Formeln): Function Points, Application Points, COCOMO
3. **Sonstige** (kreativ): Koste-es-was-wolle, Schmerzschwelle, Parkinson

7.5 Delphi-Methode

DIE STORY

Mehrere Experten schätzen ANONYM, dann diskutieren, dann wieder. Konvergiert.
Vorteil: kein dominanter Teilnehmer drückt die Schätzung in eine Richtung.

7.6 Function Points (Albrecht 1979, IBM)

MUSS-WISSEN

Aus Benutzersicht, schätzt Funktionen-Anzahl.

5 Kategorien:

- **EI** = External Input (Eingaben) $\times 3/4/6$
- **EO** = External Output (Ausgaben) $\times 4/5/7$
- **EQ** = External Inquiry (Abfragen) $\times 3/4/6$
- **ILF** = Internal Logical File (interne Daten) $\times 7/10/15$
- **EIF** = External Interface File (externe Daten) $\times 5/7/10$

Formel:

- **UFP** = Summe der gewichteten Kategorien
- **VAF** = $0,65 + 0,01 \times \text{TDI}$ (TDI aus 14 Einflussfaktoren)
- **FP** = $\text{UFP} \times \text{VAF}$

MERKHILFE

Eselsbrücke für die 5 Kategorien:

„Eingang, Ausgang, Anfrage, Innen-Daten, Außen-Daten“

→ Das deckt ALLES ab was eine App tut.

7.7 COCOMO 81 (Boehm 1981)

MUSS-WISSEN

Basis: **KDSI** (Kilo Delivered Source Instructions).

3 Projektarten (immer teurer):

- **Organic** – klein, $< 50 \text{ KDSI}$ ($\times 2.4$, Exp. 1.05)
- **Semidetached** – mittel, $< 300 \text{ KDSI}$ ($\times 3.0$, Exp. 1.12)
- **Embedded** – groß/kritisch ($\times 3.6$, Exp. 1.20)

3 Präzisionsstufen:

- **Basic** – nur KDSI
- **Intermediate** – + Korrekturfaktor M (Produkt von Mi)
- **Detailed** – noch feiner

Personenmonate (PM):

- Organic Basic: $E = 2,4 \times S^{1,05}$
- TDEV Organic: $2,5 \times E^{0,38}$

MERKHILFE

„O-S-E“ wird IMMER teurer:

2,4 < 3,0 < 3,6 (Konstanten)

1,05 < 1,12 < 1,20 (Exponenten)

Embedded ist der teuerste!

7.8 Planung: WBS, PERT, GANTT

MUSS-WISSEN

WBS (Work-Breakdown Structure) = hierarchischer Baum aller Arbeiten

PBS (PRINCE2) = wie WBS, aber für Teilprodukte

PERT = Netzplan mit Abhängigkeiten, FA/FE

GANTT = Balkendiagramm, Aktivitäten über Zeit

Burndown = verbleibender Aufwand pro Tag (agil)

MTA = Meilenstein-Trend-Analyse

CTA = Kosten-Trend-Analyse

MERKHILFE

Im PERT:

FA = Frühster Anfang = MAX(FE der Vorgänger)

FE = Frühestes Ende = FA + Dauer

7.9 Risikomanagement

MUSS-WISSEN

3 Risiko-Arten:

- **Projektrisiken** – Zeit, Budget
- **Produktrisiken** – Qualität, Leistung
- **Wirtschaftliche Risiken** – Unternehmen

4 Schritte:

1. Identifizieren (Checklisten)
2. Analysieren (Wahrscheinlichkeit + Schweregrad)
3. Planen (Vermeidung / Minimierung / Notfallplan)
4. Überwachen (Messen, Bewerten, Anpassen)

7.10 Personal-Mythos – Brooks' Law

DIE STORY

Fred Brooks (1975), „The Mythical Man-Month“:

„Adding manpower to a late software project makes it later.“

→ Mehr Leute = MEHR Verspätung (Einarbeitung, Kommunikation, Integration).

CHECK DICH SELBST

Q: Was ist der Schätztrichter?

A: Schätzungen werden über die Zeit **genauer** (kleinerer Fehler).

Q: Wie viele Function-Point-Kategorien?

A: **5** (EI, EO, EQ, ILF, EIF).

Q: Was ist der teuerste COCOMO-Projekttyp?

A: **Embedded.**

8. Konfigurationsmanagement (SCM)

IN 30 SEKUNDEN

Wie verwalte ich Änderungen, Versionen und Builds in einem Team?

3 Themen: **Änderungsmgmt, Versionsmgmt, Automatisierung.**

8.1 Änderungsmanagement

MUSS-WISSEN

Klassisch: Antrag → Analyse → Änderungskommission → Umsetzung

Agil: Änderung = User Story im Backlog

MERKHILFE

Änderungskommission betrachtet **strategisch & organisatorisch**, NICHT primär technisch:

- Konsequenzen ohne Änderung?
- Nutzen?
- Kosten?
- Wie viele Benutzer betroffen?
- Releaseplan?

8.2 Versionsmanagement – Git

DIE STORY

Git = dezentrales Versionssystem. Jeder hat ein eigenes Repository, syncs über Server.

SVN/CVS = zentral (alter Schule).

MUSS-WISSEN

Workflow:

- `clone` – Repo herunterladen
- `commit` – Änderungen lokal speichern (SHA-1 Hash)
- `push` – zum Server
- `pull` – vom Server holen + integrieren (merge!)

Mehr:

- `branch` – Verzweigung
- `tag` – Markierung (z. B. Release)

8.3 Gute Commit Messages

MUSS-WISSEN

3 Regeln:

1. Zusammenfassung & Details **trennen**
2. Zusammenfassung max. **50 Zeichen** (GitHub trennt bei 72)
3. **Referenz** zum Issue ([ISSUE-1234])

8.4 Tangled Commits vermeiden

DIE STORY

Tangled = „Verknäult“.

Ein Commit mit Typo-Fix + Bug-Fix + neuem Feature → später nicht mehr rückverfolgbar.

Best Practice: 1 Task = 1 Commit, sinnvolle Message, Issue-Ref.

8.5 Merging-Strategien

MUSS-WISSEN

- **Lock-Modify-Unlock** – Datei sperren beim Editieren (alt, langsam)
- **Copy-Modify-Merge** – jeder editiert, Konflikte beim Mergen (Git!)
- **Feature Branches** – jeder im eigenen Branch

8.6 Semantic Versioning

IN 30 SEKUNDEN

MAJOR.MINOR.PATCH (z. B. 1.3.15)

MUSS-WISSEN

- **MAJOR:** inkompatible API-Änderung
- **MINOR:** neue Funktion, abwärtskompatibel
- **PATCH:** Bug Fix, abwärtskompatibel

MERKHILFE

Postfixes:

- SNAPSHOT (Entwicklung)
- M1 (Milestone)
- RC (Release Candidate)
- GA (General Availability, public release)

8.7 Software Automation Pipeline

DIE STORY

Stell dir ein Fließband vor: *Code* → *Build* → *Unit Tests* → *Integrate* → *Integration Tests* → *Release* → *Accept Tests* → *Deploy* → *Operate*.

Jede Stufe automatisierbar.

MUSS-WISSEN

Abstufungen:

- **Build Automation** – Komponenten + Unit Tests
- **Continuous Integration (CI)** – automatisch bauen+testen+integrieren
- **Continuous Delivery** – Release bereit; Deploy **manuell**
- **Continuous Deployment** – Deploy **automatisch**
- **DevOps** – + Operations (Monitoring, Backups, Scaling)

KLAUSUR-FALLE

Delivery vs. Deployment – die Klassiker-Falle:

Delivery = Release bereit, Mensch klickt „**Deploy**“

Deployment = alles automatisch, JEDE Änderung geht in Produktion

8.8 DevOps – Wall of Confusion

DIE STORY

Spannungsfeld:

- **Dev:** „Ich will Änderungen!“
- **Ops:** „Ich will Stabilität!“

Ergebnis: „Wall of Confusion“ – Bremst die IT.

Lösung: DevOps – gemeinsame Tools, Kultur, Praktiken.

MUSS-WISSEN

DevOps Practices:

- Versionskontrolle für **alles** (auch Konfigurationen!)
- Automatisierte Tests
- Proaktives Monitoring
- CI/CD
- Container (Docker, Kubernetes)

Aber: Tools allein reichen nicht – **Kulturwandel** notwendig!

CHECK DICH SELBST

Q: Was ist der Unterschied zwischen Delivery und Deployment?

A: **Delivery** = manuelles Deploy. **Deployment** = automatisch.

Q: Wer entscheidet bei MAJOR.MINOR.PATCH?

A: **MAJOR** bei **inkompatibler API**, **MINOR** bei **abwärtskompatibler neuer Funktion**, **PATCH** bei **Bug Fix**.

Q: Wofür braucht's die Änderungskommission?

A: Strategische/organisatorische Bewertung von Änderungen (NICHT primär technisch).

9. Notfall-Cheatsheet – die Top-30 Fakten

IN 30 SEKUNDEN

Wenn du nur **EINE** Seite vor der Klausur lesen kannst – diese hier.

9.1 Zitate & Personen

MUSS-WISSEN

- **Wasserfall:** Royce 1970
- **Spiral:** (Boehm)
- **COCOMO 81:** Barry Boehm 1981 – COCOMO II: 2000
- **Function Points:** Albrecht 1979 (IBM)
- **Scrum:** Nonaka & Takeuchi (Begriff aus Rugby)
- **Brooks 1975:** „Adding manpower to a late software project makes it later.”
- **Dijkstra:** „Testing shows presence, not absence of bugs.”
- **GANTT:** Henry Gantt
- **Kano-Modell:** Kundenzufriedenheit

9.2 Schlüssel-Akronyme

MUSS-WISSEN

- **INVEST** (User Stories) = Independent, Negotiable, Valuable, Estimateable, Small, Testable
- **DoD** = Definition of Done
- **ROI** = Return on Investment
- **RE** = Requirements Engineering
- **FA / NFA** = Funktional / Nicht-Funktional
- **MVC / MVP** = Model-View-Controller / -Presenter
- **SOA** = Service Oriented Architecture
- **REST** = Representational State Transfer
- **DI / IoC** = Dependency Injection / Inversion of Control
- **AOP** = Aspect-Oriented Programming
- **CMMI** = Capability Maturity Model Integration (5 Stufen)
- **GQM** = Goal-Question-Metric
- **WBS / PBS** = Work-/Project-Breakdown Structure
- **PERT / GANTT** = Planungsdiagramme

- **MTA / CTA** = Meilenstein- / Kosten-Trend-Analyse
- **CI / CD** = Continuous Integration / Delivery / Deployment
- **SCM** = Software Configuration Management
- **POM** = Project Object Model (Maven)
- **KDSI / KSLOC** = Kilo Delivered Source Instructions / Kilo SLOC

9.3 Zahlen die du wissen MUSST

MUSS-WISSEN

- **Scrum Team:** 5–10 Personen
- **Agile Team allgemein:** 6–8 Personen
- **Sprint:** ca. 1 Monat ($\pm$ 1–2 Wochen)
- **Daily Scrum:** 15 Minuten
- **Planning Poker:** 15 Min pro Story
- **Story Points** > 13 = Epic
- **Function Points:** 5 Kategorien, 14 Einflussfaktoren
- **COCOMO:** 3 Projektarten, 3 Präzisionsstufen
- **CMMI:** 5 Stufen
- **Inspektion:** 6 Phasen
- **Anforderungen:** 9 Qualitätskriterien, 5 RE-Aufgaben
- **ISO 25010:** 8 Qualitätsfaktoren
- **Agile Manifest:** 4 Werte
- **SOAP:** Envelope, Header, Body, Fault
- **Brainstorming:** 5–10 TN, 20 Min
- **6-3-5:** 6 TN, 3 Ideen, 5 Weitergaben
- **6 Hüte:** 6 Perspektiven

10. Letzte Klausur-Tipps

IN 30 SEKUNDEN

Multiple Choice = jede Aussage einzeln prüfen, nicht überfliegen!

MUSS-WISSEN

1. **Achte auf „nicht“, „immer“, „nur“ – KILLERWÖRTER**
2. **Bei Unsicherheit:** Wähle die Antwort näher an der Foliendefinition
3. **Sehr absolut formulierte Aussagen** sind oft falsch („IMMER“, „NIE“)
4. **Begründe dir selbst** im Kopf jede Antwort
5. **Zeitmanagement:** ca. 1 Min pro Punkt
6. **Reihenfolge:** Erst easy Fragen, dann harte (markieren!)

MERKHILFE

Die häufigsten Fallen die du jetzt KENNST:

- Daily Scrum ist NICHT E-Mail-ersetzbar
- Retrospektive ist NUR Team
- Inspektion ist NICHT nur für Anforderungen
- Delivery vs. Deployment unterscheiden!
- Lasten $\neq$ Pflichtenheft
- Personentage $\neq$ Story Points
- „Es gibt KEIN bestes Prozessmodell“
- Mehr Personal $\neq$ schnelleres Projekt (Brooks)

IN 30 SEKUNDEN

Du schaffst das! Schlaf gut, frühstücke gut, geh um 16:30 in HS A und ROCK das.